Best CT0 build for the extension, and the recipe to rebuild it

Recommendation: ONNX Runtime GenAI, int4, WebGPU

This is a build of CT0 (quantization + browser port), not a new model. It's also what our browser library ct0-edge picks by default ("`auto`") on any WebGPU browser.

Build (all measured in the browser)	Size	Tool	yes/no	choice	severity ±1	s / question
genai / int4	**2.44 GB**	**official**	**83.1%**	**80.7%**	**96%**	**0.67 s**
#2 — decode-only / q4f16 (our first browser build)	2.73 GB	**manual script**	83.1%	80.7%	100%	21.2 s
#3 — WebLLM (MLC) / q4f32_1	2.37 GB	official	83.1%	78.9%	100%	5.78 s

Chrome WebGPU, Apple M2 Pro, same 202 prompts (our 172-question decision set + 30 severity cases), System-1 head readout; time = whole run ÷ prompts. yes/no = binary decision; choice = pick the right class among N options; severity ±1 = predicted 1–5 level within one of the label.

Why #1: same accuracy as #2, 30x faster, official tooling; #2 is token-by-token and built with our own quantization script. #3 has no native hidden state (needs a least-squares reconstruction) and needs one prefill per read position.

> **One free lever, any build**
>
> Asking CT0 per class (one yes/no per candidate, take the highest probability) instead of one generic "which class" question raised AUROC on 161 real vulnerable/clean code snippets (CodeQL + OWASP Juice Shop) from 0.76 to 0.83 (head.bin readout) and from 0.81 to 0.85 (logits readout) — no retraining, just the question format.

Recipe: rebuild the genai/int4 build

Run on Linux x86_64: on ARM (e.g. Apple Silicon) int4 rounding and the rope tables come out slightly different from the published files.

1. ```
   python3.12 -m venv venv && source venv/bin/activate
   pip install torch --index-url https://download.pytorch.org/whl/cpu
   pip install onnxruntime-genai==0.18.0 transformers==5.19.0 \
     numpy onnx onnx-ir==1.1.0 safetensors sentencepiece
   ```

2. ```
   hf download aliasrobotics/alias3-micro-1026 \
     --revision 61c0e7100c052a252563ceb0711d15dc58bda2d0 --local-dir ./ct0
   ```

3. ```
   G=https://gist.githubusercontent.com/eduardoworrel
   curl -O $G/ade861f0bfdb463d1e3245c15ed8fb2f/raw/build_ct0.py
   ```

4. ```
   python build_ct0.py -i ./ct0 -o ./genai-int4 -p int4 -e webgpu \
     -c ./cache --extra_options include_hidden_states=true exclude_mtp=true
   ```

Recipe: recreate `head_score` (severity calibration)

A small readout on CT0's hidden state that scores how severe a security finding is, on a 1–5 scale.
It ships as `head_score.bin`, in the same format as CT0's `head.bin`, and is used only for severity.

It is calibrated on the decode-only q4 graph, our first browser build; calibrating one per graph is also possible.

1. ```
   python3.12 -m venv venv && source venv/bin/activate
   pip install onnxruntime==1.31.0 numpy==2.5.4 transformers==5.19.0
   ```

2. ```
   mkdir -p ct0-artifacts/scripts ct0-artifacts/tok \
     ct0-artifacts/decode-only alias3-mlc
   REV=df4eddd32fb8fdb379429fa88e0969163cf26c39
   R=https://huggingface.co/eduardoworrel/alias3-micro-1026-ONNX-decode-only
   R=$R/resolve/$REV
   D=ct0-artifacts/decode-only
   curl -L -o $D/model_q4_decode.onnx $R/onnx/model_q4.onnx
   curl -L -o $D/model_q4.onnx_data $R/onnx/model_q4.onnx_data
   for f in config.json tokenizer.json tokenizer_config.json; do
     curl -L -o ct0-artifacts/tok/$f $R/$f; done
   curl -L -o alias3-mlc/head.bin $R/heads/head.bin
   ```

3. ```
   G=https://gist.githubusercontent.com/eduardoworrel
   G=$G/19d7cae3f6277f52a4bfd013b933d849/raw
   for f in three_modes.py sev_rubric.md generate_sev_cases.py \
            capture_sev_hiddens.py calibrate_score_v2.py; do
     curl -L -o ct0-artifacts/scripts/$f $G/$f; done
   ```

4. ```
   python ct0-artifacts/scripts/generate_sev_cases.py
   python ct0-artifacts/scripts/capture_sev_hiddens.py   # ~45 min, CPU (Apple M2
   Pro)
   python ct0-artifacts/scripts/calibrate_score_v2.py
   ```

   Output: `ct0-artifacts/head_score_v2.bin`.

Use it with ct0-edge (browser)

1. ```
   npm i ct0-edge@0.7.1 @huggingface/transformers "onnxruntime-web@^1.30"
   ```

2. Put the heads next to the model (outputs of both recipes):

   ```
   mkdir -p genai-int4/heads
   cp alias3-mlc/head.bin genai-int4/heads/head.bin
   cp ct0-artifacts/head_score_v2.bin genai-int4/heads/head_score.bin
   ```

   Serve `genai-int4/` next to your page (or inside the extension).

3. ```
   import { CT0 } from "ct0-edge";

   const ct0 = await CT0.load({
     genaiBaseUrl: "/genai-int4/",
     genaiModel: "model.onnx",
   });
   const q = "Is user input written into the DOM?";
   await ct0.ask({ kind: "noul", state, q });
   const options = ["dom-xss", "exfil-egress"];
   await ct0.ask({ kind: "choice", state, options });
   await ct0.ask({ kind: "score", state });   // severity 1-5 (head_score.bin)
   ```

Tested in Chrome (WebGPU) with the files from both recipes. Without `genaiBaseUrl` it loads the published files from Hugging Face; the tokenizer is fetched from the decode-only repo on Hugging Face.